

Algorithms

1. Design
2. Domain knowledge
3. Any language
4. Independent of operating system & hardware
5. Analyse

Program

- Implementation
- Programming
- prog. language
- Dependent
- Testing

Characteristics

1. Input (or more)
2. Output (must generate some output)
3. Definiteness (clarity)
4. Finiteness
5. Effectiveness (must serve some purpose)

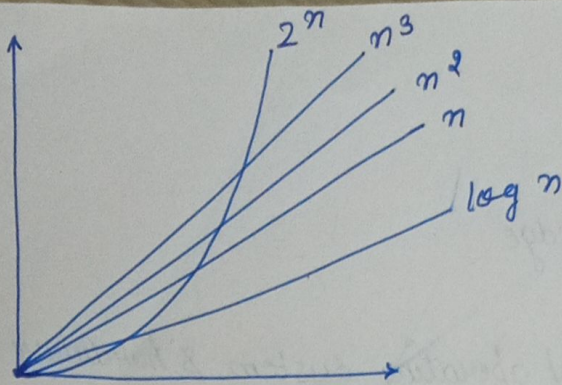
Analysing

1. Time
2. Space
3. Data transfer / Networking
4. Power
5. CPU Register

Types of time function

- $O(1) \rightarrow$ constant
- $O(\log n) \rightarrow$ logarithmic
- $O(n) \rightarrow$ linear
- $O(n^2) \rightarrow$ quadratic
- $O(n^3) \rightarrow$ cubic
- $O(2^n) \rightarrow$ exponential

$$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < \dots < 2^n < 3^n < \dots < n^n$$



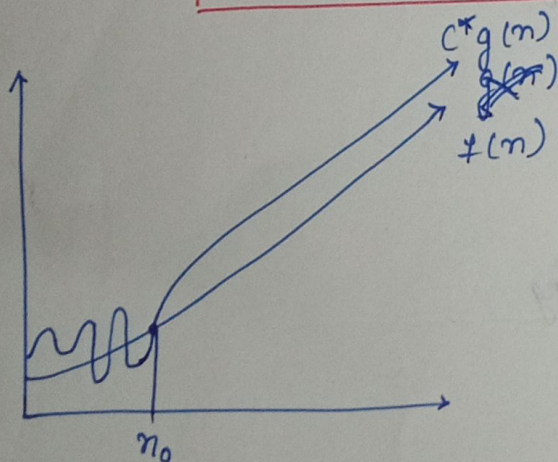
Asymptotic Notations

→ least upper bound

1. Big-oh The function $f(n) = O[g(n)]$ if $\exists$ +ve constants n_0 & c s.t

$$f(n) \leq c * g(n) \quad \forall n \geq n_0$$

$$f(n) \leq \underbrace{c * g(n)}_{\substack{\text{least upper} \\ \text{bound} \\ O[g(n)]}}$$



eg. $f(n) = 2n + 3$

$$2n + 3 \leq 10n \quad \forall n \geq 1$$

$$2n + 3 \leq 5n \quad \forall n \geq 1$$

2. $f(n) = \sum_{i=1}^n \log i \Rightarrow g(n) = n \log n \quad \forall n \geq 1$

$$\log 1 + 1$$

$$\log n!$$

$$\log n! \left[\frac{1}{n} + \frac{1}{n-1} + \dots + \frac{1}{2} \right]$$

$$n \log n \left(1 - \frac{2}{n} \right)$$

3. $2n^2 + n + 1 = f(n) \Rightarrow g(n) = 4n^2 \quad \forall n \geq 1$

4. $f(n) = \frac{n^2}{2} - 3n \Rightarrow g(n) = n^2 \quad \forall n \geq 1$

$$c = 0.46$$

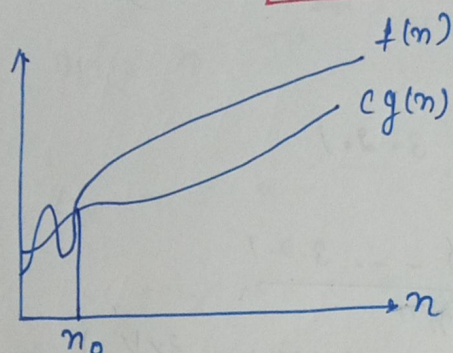
5. $3^{n+1} = O(3^n) \Rightarrow c = \frac{1}{3}$

$$f(n) \geq c * g(n)$$

6. $f(n) = a_n n^m + a_{n-1} n^{m-1} + \dots + a_0 n^0$
 $= O(n^m)$

Big omega: $\rightarrow$ The $f(n) = \Omega(g(n))$ iff $\exists$ the constant $c \neq 0$ s.t

$$f(n) \geq c \cdot g(n) \quad n \geq n_0 \text{ \& } c > 0$$



$$f(n) \geq c \cdot g(n)$$

c.g 1. $f(n) = 3n + 2 = O(n)$ $[C \leq 5, n_0 \geq 1]$

2. $f(n) = 5n^3 + 6n^2 + 3n + 2 = O(n^3)$

$$[C = 5, n_0 = 1]$$

3. $\frac{n^2}{2} - 3n = O(n^2)$, $[C \leq 0.07, n_0 \geq 1]$

$$f(n) \leq$$

$$f(n) \geq$$

Big Theta: $\rightarrow$

Let $f(n)$ & $g(n)$ be two f^m

$f(n) = \Theta(g(n))$ iff $\exists$ the constant c & n_0

$$s.t \quad [c_1 g(n) \leq f(n) \leq c_2 g(n)]$$

$\hookrightarrow$ least upper bound

$$f(n) = 3n + 2$$

greatest lower bound

$$[3n \leq 3n + 2 \leq 4n] \quad n_0 \geq 1 \text{ \& } c_1 = 4$$

$$c_2 = 3$$

$$f(n) \leq c \cdot g(n)$$

Little (o) notation: $\rightarrow$

$$f(n) = o(g(n))$$

iff $c > 0, n_0 \geq 1, n \geq n_0$ s.t $[f(n) < c \cdot g(n)]$

iff

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$$

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$$

e.g 1. $f(n) = 3n+2$
 $g(n) = n^2$ Since $\lim_{n \rightarrow \infty} \frac{3n+2}{n^2} = \frac{n^2(3/n + 2/n^2)}{n^2} \Rightarrow 0$

2. $n! \Rightarrow o(n^n)$

$f(n) = n! = n(n-1)(n-2) \dots 3 \cdot 2 \cdot 1$

$g(n) = n^n$
 $\lim_{n \rightarrow \infty} \frac{n(n-1) \dots 3 \cdot 2 \cdot 1}{n^n} = \frac{n^n (1(1-1/n)(1-2/n) \dots (1-2/n^{1/n}))}{n^n} = 0$

3. $n^k = o(2^n)$

4. $n \log n = o(n^2)$

little omega (ω)

$f(n) > c \cdot g(n)$

$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} \Rightarrow \infty$

$f(n) > \underline{g(n)}$

$\frac{f(n)}{g(n)} = \infty$

1. $n! = \omega(2^n)$

2. $\frac{n^2}{2} - \frac{n}{2}$

$C_2 = C_1 = 0.7$

$i = 1$ $S = 1$

while ($i \leq n$)

{

$i++$

$S = S + i$

pf()

}

$1 \leq n$

i	S
1	1
2	1+2=3
3	1+2+3=6
4	1+2+3+4=10

$i = 1$

$S = 1$

times

$i = 2$

$S = 1+2=3$

$i = 3$

$S = 6$

$i = k$

$S = \frac{k(k+1)}{2} = n$

$\frac{k(k+1)}{2} = n$

$O(\sqrt{n})$

$i = k$

$cg \rightarrow A(m) \{ \text{int } i, k, n$

$n+1 \rightarrow \text{for } (i=0, i \leq n, i++) \{$

$n^2 \times n \rightarrow \text{for } (j=1, j \leq i^2, j++) \{$

$n^2 \times 100 \times n \rightarrow \text{for } (k=1, k \leq 100, k++) \{$

$n^2 \times 100 \times n \rightarrow \text{--- st ---}$
 $\{$
 $\{$
 $\}$

$O(m^3)$

2.

$A(m) \{ \text{int } n = 2^{2^k}$

$\text{for } (i=1, i \leq n, i++) \{ \rightarrow n$

$\text{times}(j) \quad j=2 \quad \text{---} \rightarrow n$
 $\text{while } (j \leq n) \rightarrow n \times \underline{k+1}$
 $\{ \quad j = j^2 \rightarrow n$
 $\text{printf} ("sc65") \rightarrow (k+1) \times n$
 $\}$

$\frac{1}{2} \times n \rightarrow 2, 4 (2)$
 $\frac{2}{2} \times 16 \rightarrow 2, 4, 8 (3)$
 $\frac{3}{2} \times 256 \rightarrow 2, 4, 8, 16 (4)$

$n = 2^{2^k}$

$\log n = 2^k \log 2$

$\log \log n = \log k$

$O(n \log \log n)$

$\frac{1512}{25}$

$n+1 \leftarrow \text{for } (i=0, i \leq n, i++) \{$

$n \times n^2 \leftarrow \text{for } (j=1, j \leq i^2, j++) \{$

$n^2 \times 100 \text{ for } (k=1, k \leq 100, k++) \{$

$1 \leq 02$

254

35459

25

151

0

1

2

1

2

3

0

1

2

1

2

3

0

1

2

1

2

3

0

1

2

1

2

3

0

1

2

1

2

3

0

1

2

1

2

3

0

1

2

1

2

3

0

1

2

1

2

3

0

1

2

1

2

3

0

1

2

1

2

3

0

1

2

1

2

3

0

1

2

1

2

3

0

1

2

1

2

3

0

1

2

1

2

3

Recursion

- Big problem into smaller sub-problems
- A fn calling itself
- base condition for terminating the loop

$$T(n) = T(n-1) + 1$$

$$T(n) = T(n-2) + 1$$

$$T(n) = T(n-3) + 1$$

$$T(n) = T(n-2) + 2$$

$$T(n) = T(n-3) + 3$$

!

$$T(n) = T(n-k) + k$$

$$n-k=0 \quad \boxed{n=k}$$

$$T(n) = n \quad O(n)$$

Q $A(n)$ {

$if (n \leq 0)$

{ $print + f(" \%d ", n)$

$A(n-1)$

}

}

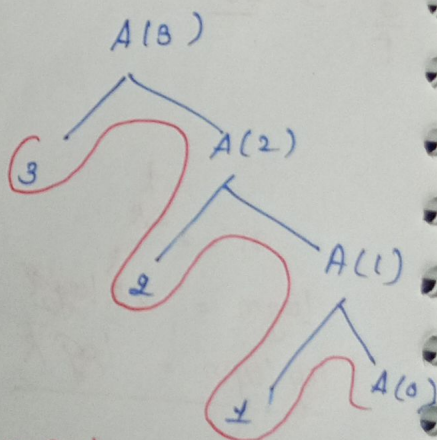
Stack size

$\Rightarrow 4$

$n+1$

$A(0)$
$A(1)$
$A(2)$
$A(3)$

for $n=3$



output $\rightarrow 3, 2, 1$

space & time $\Rightarrow O(n)$

eg. $T(n) = 2T(n-1) + 1$

$T(n-1) = 2T(n-2) + 1$

$T(n-2) = 2T(n-3) + 1$

$\rightarrow T(n-1) = 2[2T(n-3) + 1] + 1 = 4T(n-3) + 2 + 1$

$\rightarrow T(n) = 2[4T(n-3) + 2 + 1] + 1$

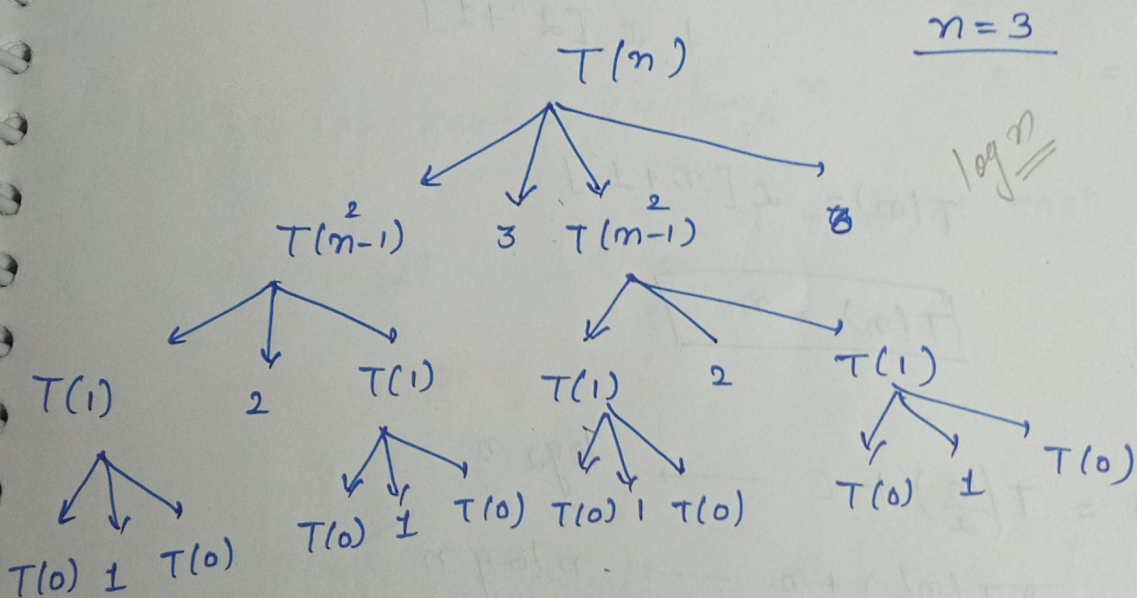
$= 8T(n-3) + 4 + 2 + 1$
 $= 2^3 T(n-3) + 2^2 + 2^1 + 2^0$

$T(n) = 2^k T(n-k) + 2^{k-1} + 2^{k-2} + \dots + 2 + 1$

$T(n) = 2^k T(n-k) + 2^{k-1} + 2^{k-2} + \dots + 2 + 1$

$T(n) = 2^k + 1$

$T(n) = 2^n = O(2^n)$



Output $\Rightarrow$ 1 2 1 3 1 2 1

Stack size $\Rightarrow$ height of tree

$$Q \quad T(n) = 2T(n/2) + C$$

$$T\left(\frac{n}{2}\right) = 2T\left(\frac{n}{2 \times 2}\right) + C$$

$$T\left(\frac{n}{4}\right) = 2T\left(\frac{n}{2^3}\right) + C$$

$$T(n/2) = 2 \left[2T\left(\frac{n}{2^3}\right) + C \right] + C \quad 2^2 T\left(\frac{n}{2^3}\right) + 2C + C$$

$$T(n) = 2 \left[2^2 T\left(\frac{n}{2^3}\right) + 3C \right] + C$$

$$T(n) = 2^3 T\left(\frac{n}{2^3}\right) + 2^2 C + 2C + C$$

$$\vdots$$

$$T(n) = 2^R T\left(\frac{n}{2^R}\right) + C [2^{R-1} + 2^{R-2} + \dots + 2 + 1]$$

$$= \quad \quad \quad + C [2^R + 1]$$

$$\frac{n}{2^R} = 1$$

$$n = 2^R$$

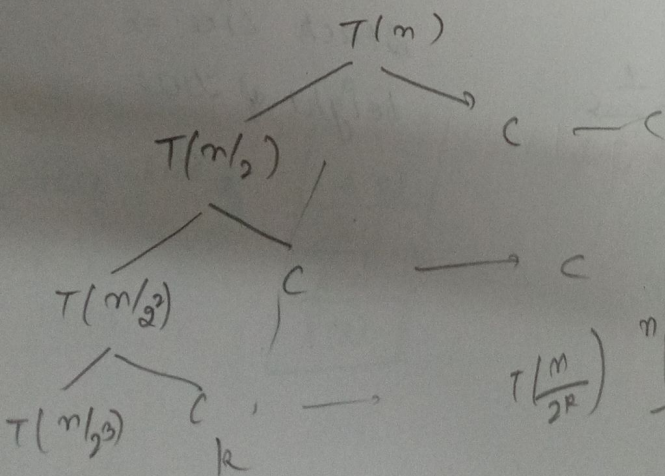
$$\log_2 n = R$$

$$T(n) = C [n + 1]$$

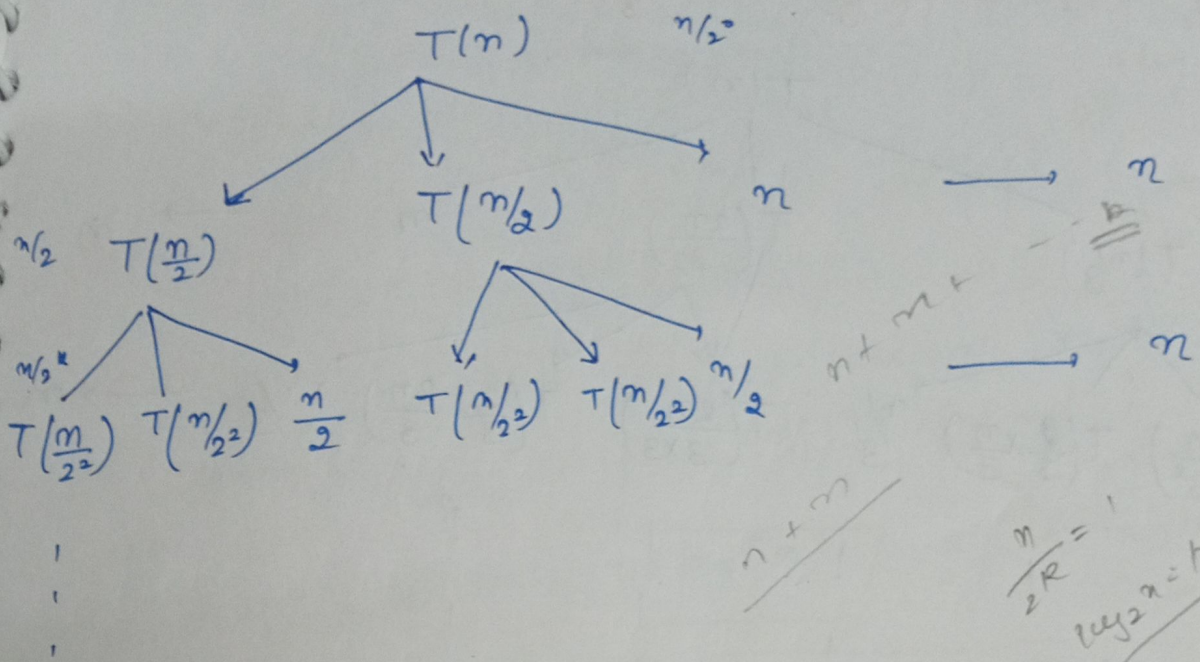
$$T(n) = n$$

$$Q \quad T(n) = T\left(\frac{n}{2}\right) + C \longrightarrow \log_2 n$$

$$Q \quad T(n) = 2T\left(\frac{n}{2}\right) + n \longrightarrow n \log n$$



Tree $\rightarrow$ a) $T(n) = 2T(n/2) + n$



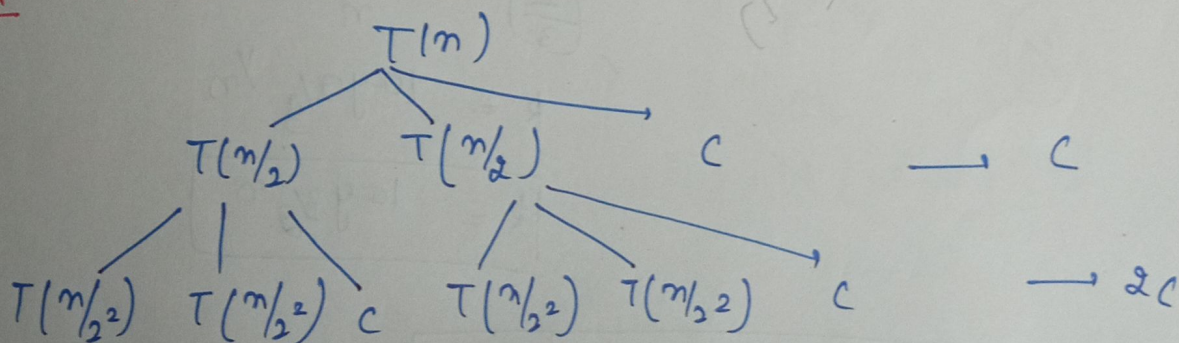
$T(n/2^k) \quad T(n/2^k) \quad \frac{n}{2^{k-1}} \quad T(n/2^k) \quad T(n/2^k) \quad \frac{n}{2^{k-1}} \rightarrow n$

Height of tree $\Rightarrow k = \log_2 n$

$\rightarrow n \log_2 n$ (point n time at each level)

Q2

$T(n) = 2T(n/2) + c$

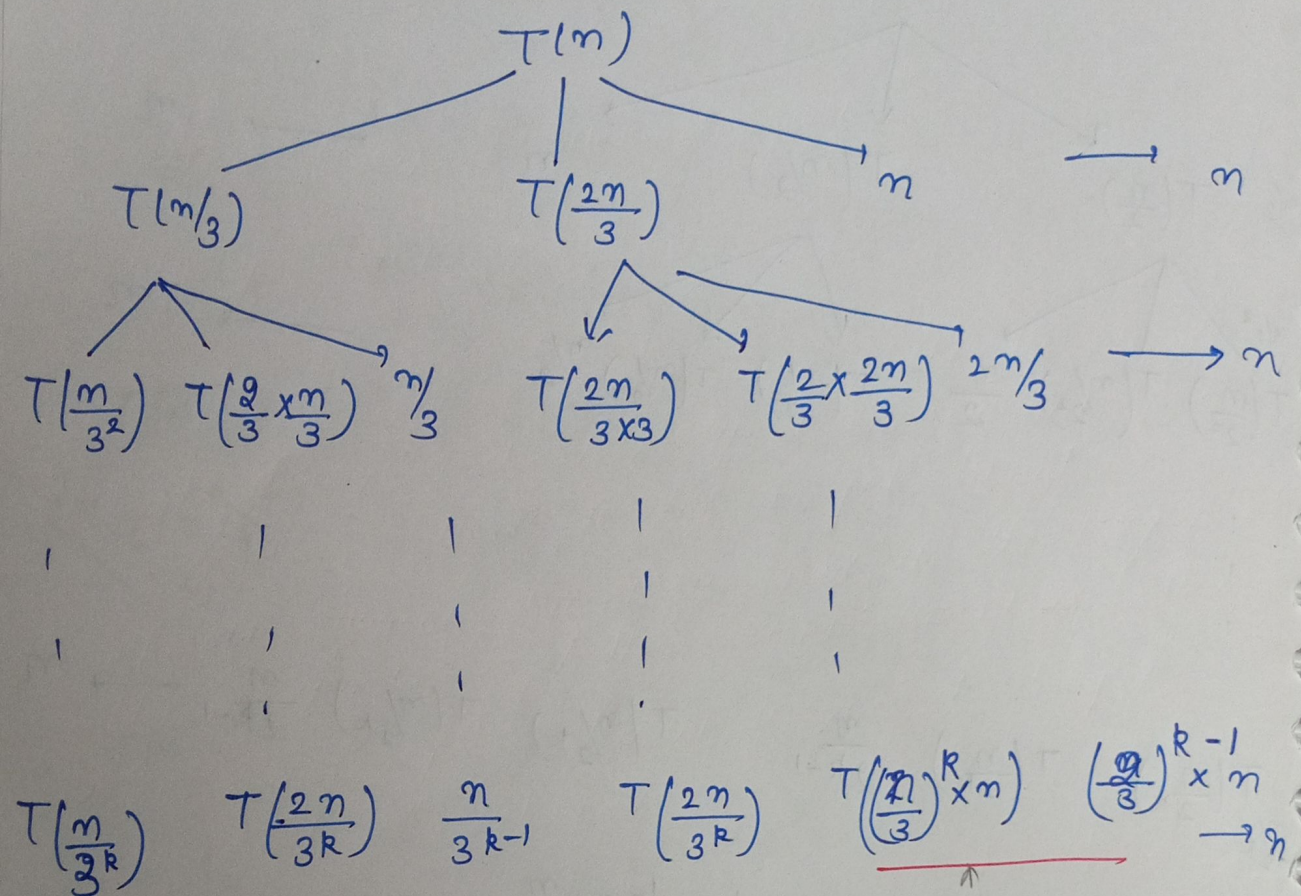


Height $\rightarrow k = \log_2 n$

$2^k \log_2 n = O(n)$

$2^{k-1}c + 2^{k-2}c + \dots + 2 + 1 = 2^k + 1$

$$Q \quad T(n) = T(n/3) + T(2n/3) + n$$



height depends on this

$$\left(\frac{2}{3}\right)^k \times n = 1$$

$$n = \log_{3/2} \frac{1}{n}$$

$$k = \log_{3/2} n$$

$$\left(\frac{2}{3}\right)^k \times n = 1$$

$$\left(\frac{2}{3}\right)^k = \frac{1}{n}$$

$$k = \log_{3/2} n$$

$$k = \log_{3/2} n$$

$$O(n) = n \times \log_{3/2} n$$

Similarity

$$T(n) = T(n/10) + T(9n/10) + n$$

$$\boxed{O(n) = n \log_{10} \frac{10}{9} \times n}$$

Master's theorem $\rightarrow$

$$T(n) = a T\left(\frac{n}{b}\right) + f(n)$$

where $a \geq 1$, $b > 1$ are two constants & $f(n)$ asymptotically $\approx$

$$\boxed{T(n) = a T(n/b) + \theta(n^k \log^p n)}$$

$$a > b^k \quad a T(n/b) + \theta(n^k \log^p n)$$

$$a > b^k \quad \frac{p > -1}{n^{\log_b a} \log^{p+1} n}$$

Case I $\rightarrow$ If $a > b^k$ then

$$\boxed{T(n) = \theta(n^{\log_b a})}$$

Case II $\rightarrow$

$$\text{If } a = b^k$$

$$1. \text{ If } p > -1, \text{ then } T(n) = \theta(n^{\log_b a} \times \log^{p+1} n)$$

$$2. \text{ If } p = -1, \text{ then } T(n) = \theta(n^{\log_b a} \times \log \log n)$$

$$3. \text{ If } p < -1, \text{ then } T(n) = \theta(n^{\log_b a})$$

Case III $\rightarrow$

$$a < b^k$$

$$(a) \text{ If } p \geq 0, \text{ then } T(n) = \theta(n^k \log^p n)$$

$$(b) \text{ If } p < 0, \text{ then } T(n) = \theta(n^k)$$

$$\frac{n^k \log^p n}{n^k}$$

$$\frac{n^k \log^p n}{n^k}$$

SEARCHING ALGOS

efficiently finding any element from array.

Types

1. Linear search (sequential search)

2. Binary search

Linear search $\rightarrow$ we iterate over all the elements of the array and check if the current element is equal to the target element

2 $\rightarrow$ If the current element is equal to the target element, then index of the current element is returned

3 $\rightarrow$ otherwise, if no element is equal to target then -1 is returned.

Algo $\rightarrow$

LinearSearch (int arr[], int N, int x)

```
{
  for (i=0; i < N; i++) {
    if (arr[i] == x)
      return i;
  }
```

```
  return -1;
}
```

Iterative

LS (array, index, key):

```
if index < 0;
```

```
  return -1
```

```
if item index = key
```

```
  return index
```

```
  return LS (array, index-1, key)
```

Recursive

Time Complexity :- $\rightarrow$

Worst case $\rightarrow \underline{O(N)}$

Best case $\rightarrow \underline{O(1)}$

Avg $\rightarrow \underline{O(N)}$

Space $\rightarrow O(1)$

Binary Search :- $\rightarrow$ It is a search algorithm used to find the position of a target value within a sorted array.

It works by repeatedly dividing the search interval in half until the target value is found or interval is empty.

The search interval is halved by comparing the target element with middle value of search space.

Conditions :- $\rightarrow$

$\rightarrow$ The data structure must be sorted.

$\rightarrow$ Access to any element of data structure should be constant time.

Iterative :- $\rightarrow$

```
int BinarySearch (A, n, key)
```

```
{  
    l = 1, h = n  
    while (l <= h)
```

```
{  
    mid =  $\frac{l+h}{2}$ ;
```

```
    if (key == A[mid])
```

```
        return mid;
```


if (key < A[mid])
h = mid - 1;

else
l = mid + 1;

}
return 0;

}

T.C →

Min → $O(1)$

Max → $O(\log n)$

Arg → $O(\log n)$

Recursive: →

Algo RBIS(l, h, key, A)

{ if (l == h)

{ if A[l] == key]

return l;

else

return 0

}

else

mid = $\left\lfloor \frac{l+h}{2} \right\rfloor$

if [key == A[mid]]

return mid;

if [key < mid)

return RBIS(l, mid-1, key)

else

return RBS (mid+1, h, rex);

}

}

Sorting: →

Reduce the searching time

Classificatⁿ: →

1. No. of comparisons
2. No. of swaps
3. By memory usage
4. By recursion
5. By stability
6. Adaptivity
7. Inplace / Outplace

$$\text{odd} = \frac{3(n-1)}{2} + 1$$

$$\text{even} = \frac{3(n-2)}{2} + 1$$

Q Algo to find min. comparison to find smallest & largest element in an array.

Small, large (first two elements) → compare them → If elements are even

for (i = 0/2, i ≤ n-1, i = i+2) {

If (A[i] < A[i+1])

{ If (A[i] < small)

small = A[i];

If (A[i+1] > large)

large = A[i+1]

}

else {

If (A[i+1] < small)

small = A[i+1]

If (A[i] > large)

large = A[i]

}

4	3	2	7	6	8
---	---	---	---	---	---

i=2, i=4

0 1 2 3 4

S = 1 → If elements are odd

$$\boxed{\begin{matrix} l = 8 \\ S = 2 \end{matrix}}$$

odd i = 1

$$\text{Comparison} = \frac{3(n-1)}{2}$$

even i = 2

$$= \frac{3(n-2)}{2} + 1$$

Depending upon odd or even No. of elements in array

Selection Sort: →

1. Find the smallest element & swap it with the first element.
2. Then find smallest among remaining element.
3. We keep doing it until we get all elements moved to correct position.

Algo: →

Algo

```

selectionSort (int arr[], int n) {
    for (i = 0, i < n, i++) {      → n+1
        int min_ind = i
        for (j = i+1, j < n; j++) { → n
            if (arr[j] < arr[min_ind]) {
                min_ind = j
            }
        }
    }
}

```

finding min
in unsorted
array

```

int temp = arr[i];
arr[i] = arr[min_ind]
arr[min_ind] = temp;

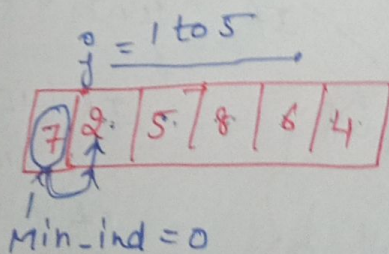
```

Swap

3

S = 6	B = 6	L = 6
NR	O(N ²)	O(N)

min_ind = 2



i = 0 to 5

Comparisons $\rightarrow N^2$ $\rightarrow \frac{n(n-1)}{2}$

Swaps $\rightarrow$ best case (sorted array) = 0

worst case (unsorted array) = $N-2 / N-1$

Time Complexity $\rightarrow$

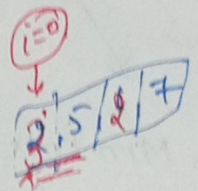
best case $\rightarrow O(n)$

Worst $\rightarrow O(n^2)$

Memory $\rightarrow O(1)$

Not stable $\rightarrow$ it changes the relative order of equal elements

Adaptivity $\rightarrow x$



Recursive $\rightarrow$

Min Index ($a[i]$, i , j)

if ($i=j$)

return i ;

int $k = \text{minIndex}(a, i+1, j)$;

return ($a[i] < a[k]$) ? i : k ;

}

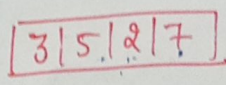
$O(n^2)$
 $O(n)$ $\rightarrow$ space

(a, i, n)

$k = (a, i+1, n)$

$k = a[i+1, n]$

(1) (2)



SelectⁿSort ($a, n, i_{\text{dex}}=0$)

if (index == n)

return;

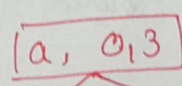
$k = \text{minIndex}(a, \text{index}, n-1)$;

if ($k \neq \text{index}$)

swap($a[k], a[\text{index}]$)

SelectⁿSort ($a, n, \text{index}+1$)

}



$a[0] <$

$k = 0, 1, 3$

$k = 0, 2, 3$ $i=2$ $a[2] <$ $i=2$

$k = 0, 3, 3$ $i=3$ $a[3] <$ $a[2]$

$a[2] < a[3]$

Insertion Sort :-

Iteratively inserting each element of an unsorted list into its correct position in a sorted portion of the list.

- Start with 2nd element of the array, assuming that the first is sorted.
- Compare second element with the first & check if the 2nd element is smaller then swap.
- Move to third & compare it with the first two & put it in correct position.

Algo :-

Insertion Sort (int a[], int n)

for (i = 1 to n, i++) {
 $O(n)$ key = a[i]
 $j = i - 1$

while (j >= 0 && a[j] > key)

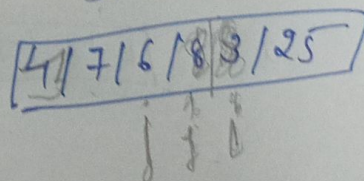
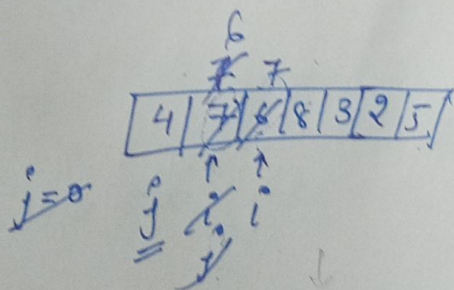
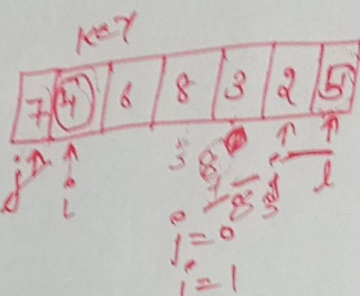
{
~~a[j+1] = a[j]~~ $\rightarrow O(n)$

j = j - 1 ; j--

}
a[j+1] = key

$O(n^2)$

NO EXCESS IN B.C



Comparison $\rightarrow 1 + 1 + \dots + (n-1) \Rightarrow \frac{n(n-1)}{2} = O(n^2)$

Swap $\rightarrow$ best case $\rightarrow 0$
worst $\rightarrow N-1$

Stable, Adaptability, Inplace.

Time Complexity
 $\rightarrow$
best case $\rightarrow O(n)$
worst $\rightarrow O(n^2)$

Recursive $\rightarrow$ InsertSort(A[], n) $\rightarrow T(n)$

if (n <= 1)
return

$T(n-1) \rightarrow$ IS(A[], n-1)

last = A[n-1]
j = n-2

while (j >= 0 && A[j] > last) {

A[j+1] = A[j]
j--;

}

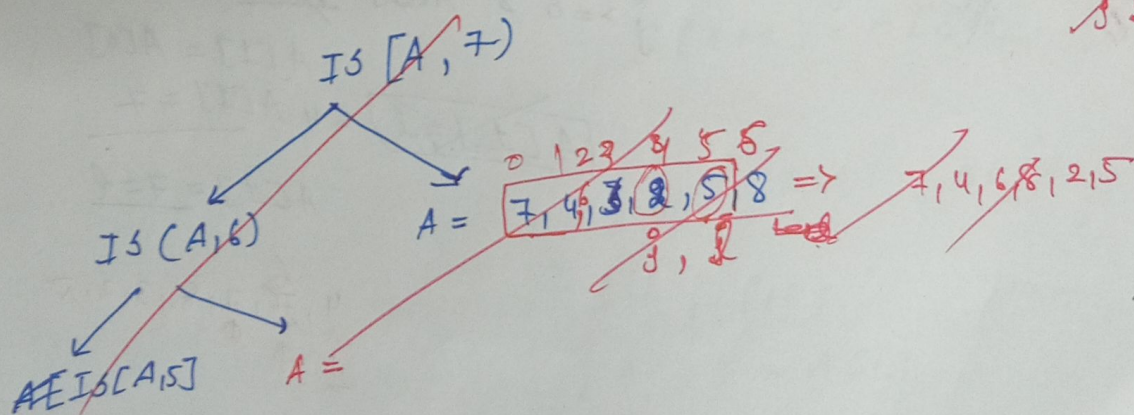
A[j+1] = last;

}

$T(n) = T(n-1) + n$
 $O(n^2)$

$O(n)$

A = ~~7~~, 4, 6, 8, 3, 5
7, 4, 6, 8, 2, 5



Time Complexity $\rightarrow$

Best Case $\rightarrow O(n)$

Worst Case $\rightarrow O(n^2)$

Recursive $\rightarrow$ $T(n) = T(n-1) + n \rightarrow O(n^2)$
 $\hookrightarrow$ best $\rightarrow$ No need to swap

$$\boxed{T(n) = T(n-1) + 1} \rightarrow O(n)$$

Merge Sort $\rightarrow$

Dividing the input array into smaller subarrays & sorting these subarrays, then merging them

Array left middle right
 $\downarrow$ $\swarrow$ $\searrow$ $\rightarrow$

algo $\rightarrow$

Merge (A, B, l, m, r)

x, l, m, r

$\hookrightarrow m = m - l + 1$

$n = r - m$

A[n1], B[n2]

Merge (A, B, m, n)

i, j, r = 0

while (i <= m && j <= n)

if (A[i] < B[j])

C[r++] = A[i++]

else

C[r++] = B[j++]

No
initialization

for remaining
elements

$\left\{ \begin{array}{l} \text{for } (i = 0, i <= m, i++) \\ \quad C[r++] = A[i] \\ \text{for } (j = 0, j <= n, j++) \\ \quad C[r++] = B[j] \end{array} \right\}$

Merge Sort (A, l, h) $\rightarrow T(n)$

{ if ($l < h$)

{ ~~m~~ mid = $\frac{l+h}{2}$

Merge Sort (l, mid)
 " ($A, mid+1, h$)

← Merge (~~l, mid, h, n~~)
 A, B, m, n

}

Recursive

→ $T(n/2)$
 → $T(n/2)$
 → $O(n)$

$A[m-l+1] =$
 $B[\text{---} - m]$

Ⓐ Ⓑ

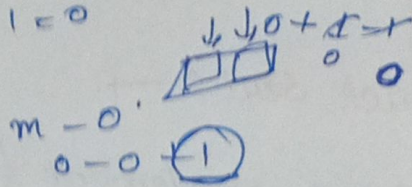
$A[10-0+1] = 4$
 $B[11-11] = 6$

$$n1 = 0 \rightarrow 0 + 0 = 0$$

$$0 - 1 + 1 = 0 \quad n1 = 0$$

$$n2 = 1 - 0 = 1$$

$$\rightarrow L[0] \rightarrow R[0] = A[0+0] = A[0] = 4$$



$$n1 = 0 - 0 + 1$$

$$n2 = 1 - 0 = 1$$

$$A[1] =$$

$$B[1] =$$

$$A[0] = Arr[0+0] = Arr[0] = 4$$

$$A[0] = Arr[0+0] = Arr[1] = 6$$

$$A[i] < B[i]$$

$$arr[0] = A[i]$$

$$arr[0] = 4$$

4, 6

$$arr[1] = 6$$

copying elements
 $A[n1]$, $B[n2]$

$$n1 = m - 1 + 1$$

$$n2 = 1 - m$$

Time Complexity $\rightarrow$

$$\text{Recursive} \rightarrow 2T(n/2) + O(n)$$

$$= O(n \log n) \quad (\text{same for iterative})$$

$$\text{Space} \rightarrow O(n) \rightarrow \text{for temporary arrays}$$

$$0 + 1 = 1$$

$$1 + 1 = 2$$

$$2 + 1 = 3$$

$$3 + 1 = 4$$

$$4 + 1 = 5$$

$$5 + 1 = 6$$

$$6 + 1 = 7$$

$$7 + 1 = 8$$

$$8 + 1 = 9$$

$$9 + 1 = 10$$

$$10 + 1 = 11$$

$$11 + 1 = 12$$

$$12 + 1 = 13$$

$$13 + 1 = 14$$

$$14 + 1 = 15$$

$$15 + 1 = 16$$

$$16 + 1 = 17$$

$$17 + 1 = 18$$

$$18 + 1 = 19$$

$$19 + 1 = 20$$

$$20 + 1 = 21$$

$$21 + 1 = 22$$

$$22 + 1 = 23$$

$$23 + 1 = 24$$

$$24 + 1 = 25$$

$$25 + 1 = 26$$

$$26 + 1 = 27$$

$$27 + 1 = 28$$

$$28 + 1 = 29$$

$$29 + 1 = 30$$

$$30 + 1 = 31$$

$$31 + 1 = 32$$

$$32 + 1 = 33$$

$$33 + 1 = 34$$

$$34 + 1 = 35$$

$$35 + 1 = 36$$

$$36 + 1 = 37$$

$$37 + 1 = 38$$

$$38 + 1 = 39$$

$$39 + 1 = 40$$

$$40 + 1 = 41$$

$$41 + 1 = 42$$

$$42 + 1 = 43$$

$$43 + 1 = 44$$

$$44 + 1 = 45$$

$$45 + 1 = 46$$

$$46 + 1 = 47$$

$$47 + 1 = 48$$

$$48 + 1 = 49$$

$$49 + 1 = 50$$

$$50 + 1 = 51$$

$$51 + 1 = 52$$

$$52 + 1 = 53$$

$$53 + 1 = 54$$

$$54 + 1 = 55$$

$$55 + 1 = 56$$

$$56 + 1 = 57$$

$$57 + 1 = 58$$

$$58 + 1 = 59$$

$$59 + 1 = 60$$

$$60 + 1 = 61$$

$$61 + 1 = 62$$

$$62 + 1 = 63$$

$$63 + 1 = 64$$

$$64 + 1 = 65$$

$$65 + 1 = 66$$

$$66 + 1 = 67$$

$$67 + 1 = 68$$

$$68 + 1 = 69$$

$$69 + 1 = 70$$

$$70 + 1 = 71$$

$$71 + 1 = 72$$

$$72 + 1 = 73$$

$$73 + 1 = 74$$

$$74 + 1 = 75$$

$$75 + 1 = 76$$

$$76 + 1 = 77$$

$$77 + 1 = 78$$

$$78 + 1 = 79$$

$$79 + 1 = 80$$

$$80 + 1 = 81$$

$$81 + 1 = 82$$

$$82 + 1 = 83$$

$$83 + 1 = 84$$

$$84 + 1 = 85$$

$$85 + 1 = 86$$

$$86 + 1 = 87$$

$$87 + 1 = 88$$

$$88 + 1 = 89$$

$$89 + 1 = 90$$

$$90 + 1 = 91$$

$$91 + 1 = 92$$

$$92 + 1 = 93$$

$$93 + 1 = 94$$

$$94 + 1 = 95$$

$$95 + 1 = 96$$

$$96 + 1 = 97$$

$$97 + 1 = 98$$

$$98 + 1 = 99$$

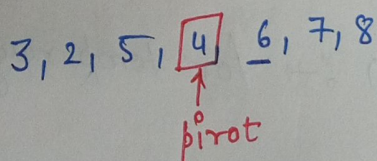
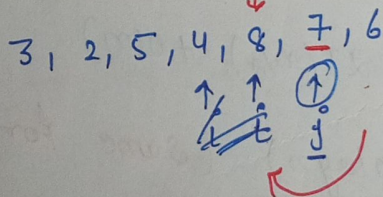
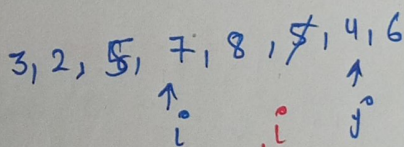
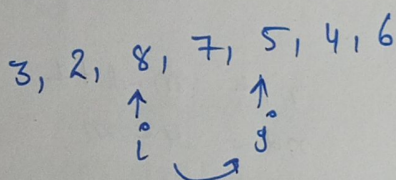
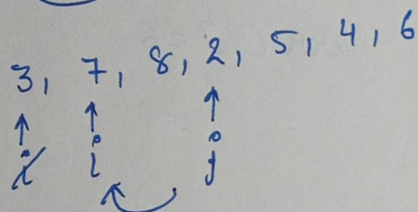
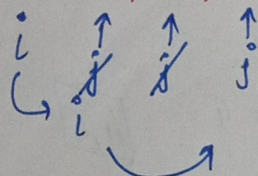
$$99 + 1 = 100$$

stable

Quick Sort: $\rightarrow$

1. Select an element from the array as pivot.
2. Rearrange the array around the pivot. i.e. pivot on its correct position.
3. Apply same procedure on R & L of pivot.

c.g. $8, 7, 3, 2, 5, 4, \boxed{6}$ ^{pivot}



- $\rightarrow$ assign i & j
- $\rightarrow$ Increment i until element greater than pivot
- $\rightarrow$ Decrement j until element less than pivot
- $\rightarrow$ exchange them

Algo: →

partition (A, p, r)

{

x = A[r]

i = p - 1

for (j ← p to r - 1)

{

If (A[j] ≤ x)

{ i = i + 1

Swap (A[j], A[i])

}

j ←

Swap (A[i + 1], x)

return i + 1

}

If half ⇒ T(n) = 2T(n/2) + O(n)
O(n log n)

Quick sort (A, p, r)

{

If (p < r)

{

q = partition (A, p, r)

Quicksort (A, p, q - 1)

(A, q + 1, r)

}

}

Depends on this partition

Divide array into half

Time complexity: →

Best case → O(n log n)

Avg → O(n log n)

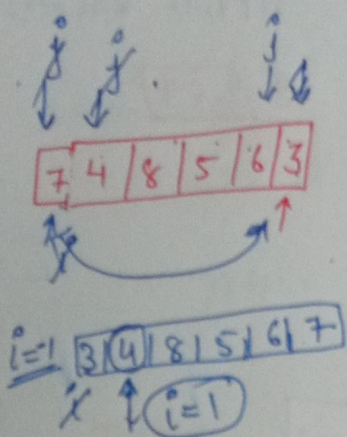
Worst → O(n²)

unsorted array worst case

(7, 6, 5, 4, 3, 2, 1)

(1, 2, 3, 4, 5, 6, 7)

When smallest & largest element choose as pivot



Heapsort :- $\rightarrow$

- $\rightarrow$ It is a comparison based sorting based on BT,
- $\rightarrow$ Optimized over selection sort

1. first convert the array into max heap using heapify
2. happens in place
3. one by one delete the root node of Max-heap & replace it with the last node & heapify.

Types $\left[\begin{array}{l} \rightarrow \text{Min heap} \\ \rightarrow \text{Max heap} \end{array} \right.$

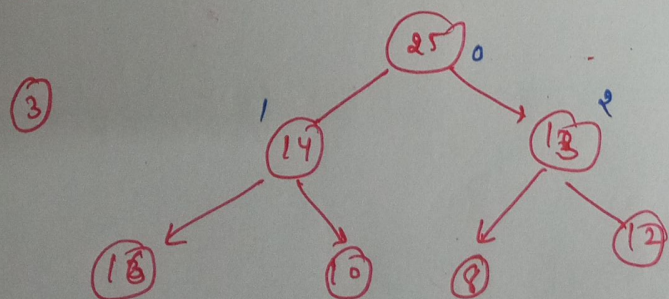
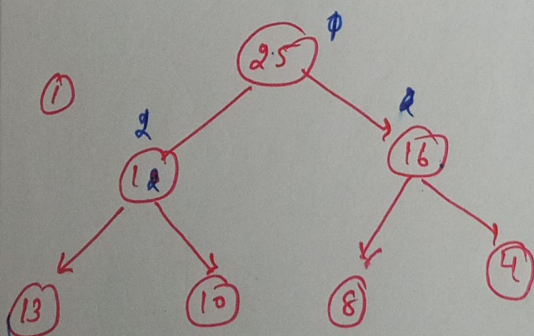
$n = 2^{h+1} - 1$ $n \rightarrow \text{nodes}$
 $h \rightarrow \text{height}$

how to find the heap size?

① 25, 12, 16, 13, 10, 8, 4

② 25, 10, 16, 13, 10, 8, 12

n	Heap size
7	2
7	7
7	2



$\left\lfloor \frac{n}{2} \right\rfloor + 1 \text{ to } n \Rightarrow \text{leaf nodes}$

$\left(\frac{n}{2} \right) \text{ to down } n \rightarrow$

Internal nodes

Algo :-

Max-heapify (A, i) (A, n, i)

{

$$l = 2 * i$$

$$r = 2 * i + 1$$

If ($l \leq A.heapsize$ && $A[l] > A[i]$)

largest = l;

else

largest = i

If ($r \leq A.heapsize$ && $A[r] > A[largest]$)

largest = r

If (largest $\neq$ i)

{ exchange ($A[i]$, $A[largest]$)

Max-heapify ($A, largest$)

}

}

Build-Max-heap (A, n) {

$n/2 \rightarrow$ for ($i = \lfloor n/2 \rfloor$ down to 1) {

$\rightarrow$ Max-heapify (A, i)

}

HeapExtractHeap(A) {

Max = $A[1]$

$A[1] = A[A.heapsize]$

$A.heapsize = A.heapsize - 1$

Max-heapify ($A, 1$)

} Return Max

No Need

$T(n) \leq T(n/3) + O(1)$
 $\rightarrow$ size of subtree
 $\rightarrow O(\log n)$
 $O(h) \Rightarrow$ height of tree

extract^m

Heapsort (int A, int n)

$n/2 \rightarrow$ Build-Max heap; $\rightarrow n \log n$

$n \rightarrow$ for ($i = \underline{n-1}$, $i > 0$, $i--$)

{ swap (A[i], A[0])

$n \rightarrow$ heapify (A, $\underline{i}$, 0);

$n \log n \rightarrow$ H.H (A, 0)

$O(n \log n)$

14 | 8 | 1

↑

7 | 8 | 1
0 1 2

Time complexity $\rightarrow O(n \log n)$

time taken by heapify $\rightarrow$

$$\text{Total nodes at } h = \left\lceil \frac{n}{2^{h+1}} \right\rceil$$

$$T(n) = \sum_{h=0}^{\log n} \left\lceil \frac{n}{2^{h+1}} \right\rceil \times O(h) \quad \rightarrow \text{each call of max-heap at each node.}$$

$$= \frac{cn}{2} \sum_{h=0}^{\log n} \left(\frac{h}{2^h} \right)$$

$$= \frac{cn}{2} \times 2 = cn$$

$$\frac{cn}{2} \times 2 = cn$$

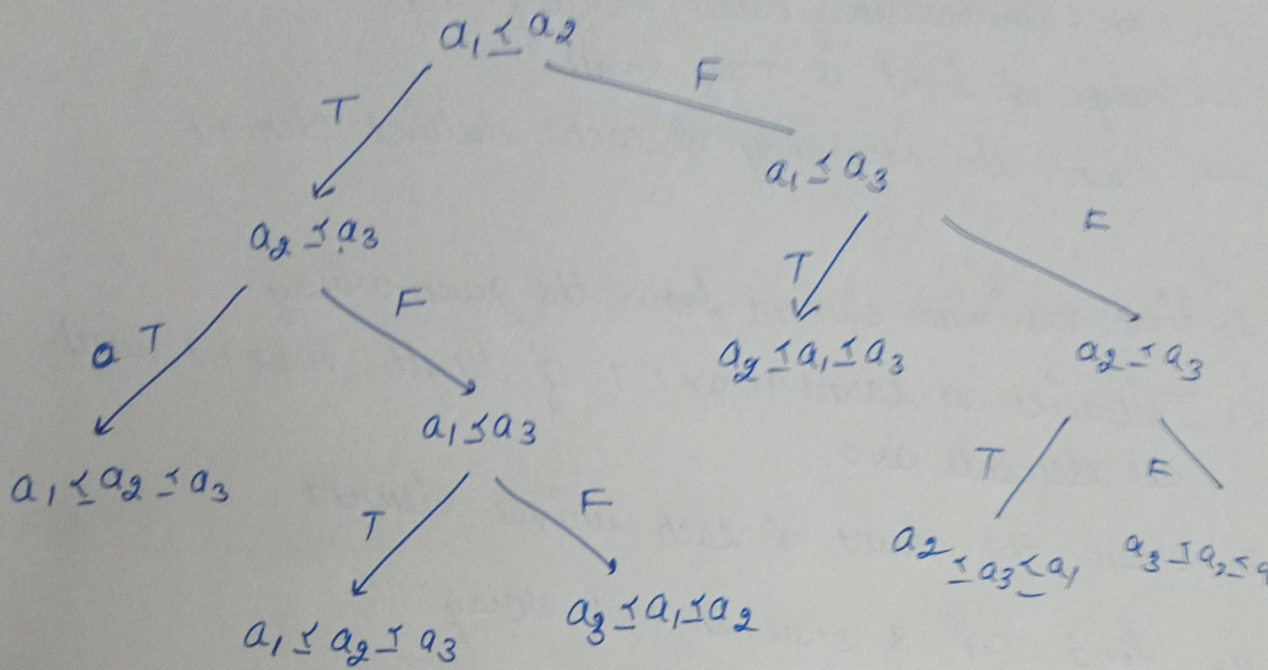
$$\boxed{T(n) = O(n)}$$



Max-heapify

Lower Bound Sorting $\rightarrow$

a_1, a_2, a_3



$$n! \leq 2^h$$

$$\log n! \leq h \log 2$$

$$h \geq \log n! = n \log n$$

$$h \geq n \log n$$

$$h = \Omega(n \log n)$$

Combination based
Sorting algo.

Count Sort $\rightarrow$

- $\rightarrow$ Non-comparison based sorting algo.
- $\rightarrow$ Range of input is very small
- $\rightarrow$ Count the frequency of each distinct element

1. Find maximum element from the given array.
2. Initialize a countArray[] of length $\text{max}+1$ with all elements as 0.
3. Store the count of each unique element.
4. Find C.O.F & store in that array
5. Iterate from the end of the input array
6. update output array $[\text{countA}[\text{I} \cdot \text{A}[i]-1] = \text{In} \cdot \text{A}[i]]$

Algo

$O(n)$

Count Sort (A, B, k)

{

$C = [0, \dots, k]$

for $i = 0$ to k

$C[i] = 0$

for $j = 1$ to n

$C[A[j]] = C[A[j]] + 1$

for $i = 1$ to k

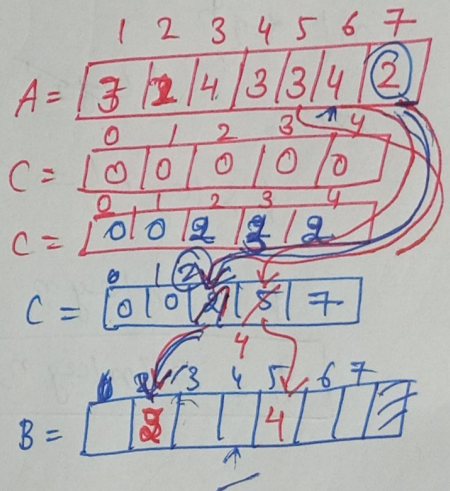
$C[i] = C[i] + C[i-1]$

for $j = n$ down to 1

{ $B[C[A[j]]] = A[j];$

$C[A[j]] = C[A[j]] - 1$

}



$\frac{C[3]}{0+1}$

$C[2]$

$O(N+M)$

Multiplicatⁿ Algo: $\rightarrow$

$$a = a_1 \times 10^{n/2} + a_2$$

$$b = b_1 \times 10^{n/2} + b_2$$

$$a \times b = \underbrace{(a_1 \times b_1)}_A \times 10^n + \underbrace{[a_1 \times b_2 + b_1 \times a_2]}_{B+C} \times 10^{n/2} + \underbrace{a_2 \times b_2}_D$$

$$= A \times 10^n + (B+C) \times 10^{n/2} + D$$

Algo: $\rightarrow$

Multiply(a, b)

{

Assume $n = \text{length}(a) = \text{length}(b)$

If $\text{length}(a) \leq 1$

return $a \times b$

partition a, b into $a = a_1 \times 10^{n/2} + a_2$
 $b = b_1 \times 10^{n/2} + b_2$

$$A = \text{Mul}(a_1, b_1)$$

$$B = \text{Mul}(a_1, b_2)$$

$$C = \text{Mul}(a_2, b_1)$$

$$D = \text{Mul}(a_2, b_2)$$

return $A \times 10^n + (B+C) \times 10^{n/2} + D$

$$T(n) = 4T(n/2) + n$$

$$A = \text{Mul}(a_1, b_1)$$

$$B = \text{Mul}(a_2, b_2)$$

$$C = \text{Mul}(a_1, b_2)$$

$$D = \text{Mul}(a_2, b_1)$$

$$CD = \text{Mul}(a_1 + a_2, b_1 + b_2)$$

$$A = \text{Mul}(a_1, b_1)$$

$$B = \text{Mul}(a_2, b_2)$$

$$CD = \text{Mul}(a_1 + a_2, b_1 + b_2)$$

$$\text{Return} = A \times 10^m + (CD - A - B) \times 10^{m/2} + B$$

$$T(n) = \Theta(n^{\log_2 3})$$

Randomised Select

Random(int arr[], int low, int high)

{

Random rand = new Random();

int pivot = rand.nextInt(high - low + 1) + low;

Swap(arr, pivot, high)

// call partition

}

or return

Index pivot

Randomized Select

Randomized-Select (A, p, r, i)

If ($p == r$)
return $A[p]$

$q = \text{randomized-partition}$

$k = q - p + 1$

If ($i == k$)
return $A[q]$

else if ($i < k$)
return $\text{R-Select}(A, p, q-1, i)$

else
return $\text{R-Select}(A, p+1, r, i-k)$

}

partition algo

init